# Unit 02: Installation, Boot Process, and User Administration

## 2.1 Linux Installation and Boot Process

### Linux Installation

- **Methods of Installation:**
    - **Local Installation:** Using CD/DVD or USB drive.
    - **Network-Based Installation:** Download packages during installation from a network server.
- **Steps for Installation:**
    - Boot from installation media (USB/CD).
    - Choose language, time zone, and keyboard layout.
    - Partition disks (Manual or Guided).
    - Select software packages and desktop environment (if required).
    - Configure user accounts and passwords.
    - Complete installation and reboot.

### Boot Sequence

The boot sequence is the series of steps the system follows to start the computer, initialize hardware, and load the operating system. Understanding it is crucial for troubleshooting startup issues and managing system services.

1. **BIOS/UEFI Initialization** – Hardware detection and POST (Power-On Self Test).
2. **Boot Loader Stage** – Loads the Linux kernel into memory.
3. **Kernel Initialization** – Kernel initializes hardware and mounts the root filesystem.
4. **Init/Systemd Process** – Starts system services and user-space processes.
5. **Login Prompt / GUI** – Presents the user login interface.

### Boot Loaders

A boot loader is a small program that loads the operating system kernel into memory and starts the boot process. It is executed after the BIOS/UEFI has initialized the

hardware. Boot loaders are essential because the BIOS/UEFI itself cannot understand the filesystem or load an OS directly.

- **GRUB (GRand Unified Bootloader)** is the most common bootloader.
- **Responsibilities:**
    - Select the OS to boot.
    - Load kernel and initramfs.
    - Pass parameters to the kernel.
- **Common GRUB Commands:**

```
grub-install /dev/sda          # Install GRUB
update-grub                    # Update GRUB configuration
```

### Kernel Modules

Kernel modules are separate, modular pieces of code that can be dynamically loaded into or removed from the Linux kernel. They extend the kernel's functionality without requiring a reboot, providing support for additional hardware, filesystems, or system features.

- **Commands to Manage Modules:**

```
lsmod                          # List loaded modules
modprobe module_name           # Load module
rmmod module_name              # Remove module
```

- **Configuration:** Modules can be loaded at boot via /etc/modules-load.d/ or modprobe.d/.

## 2.2 User Account Management and Security

### User Account Management

- **Add a user:**

```
sudo useradd username
sudo passwd username                    # Set password
```

- **Modify a user:**

```
sudo usermod -aG groupname username     # Add to a group
```

- **Delete a user:**

  sudo userdel username
  sudo userdel -r username  # Delete home directory too

**Group Administration**

- **Add a group:**

  sudo groupadd groupname

- **Modify a group:**

  sudo groupmod -n newname oldname

- **Delete a group:**

  sudo groupdel groupname

- **View groups of a user:**

  groups username

**Password Policies**

- **Set password expiration and complexity:**

  sudo chage -l username  # View password info
  sudo chage -M 90 username  # Max password age 90 days

- Configure /etc/login.defs or PAM modules for:
  - Minimum password length
  - Password history
  - Lockout policy

**Authentication Configuration**

Authentication configuration in Linux refers to the setup of mechanisms that verify the identity of users or services before granting access to the system or resources. Proper authentication ensures that only authorized users can log in, execute commands, or access files.

- Managed using Pluggable Authentication Modules (PAM).
- **Configuration files:** /etc/pam.d/

- Enables:
    - Local authentication
    - LDAP, Kerberos, or Active Directory authentication

**File Permissions**

- **Permissions Types:**
    - Read (r), Write (w), Execute (x)
- **Commands:**

```
chmod 755 file.txt              # Change permissions
chown user:group file.txt       # Change owner and group
```

- **Special Permissions:**
    - **Setuid (s):** Execute as file owner
    - **Setgid (s):** Execute as group owner
    - **Sticky bit (t):** Only file owner can delete in shared directory

**Access Control Lists (ACLs)**

Access Control Lists (ACLs) provide fine-grained permission control over files and directories beyond the traditional rwx (read, write, execute) permissions. ACLs allow you to specify different permissions for multiple users and groups on the same file or directory, which is not possible with standard Unix permissions alone.

- **Commands:**

```
getfacl file.txt                # View ACL
setfacl -m u:user:rwx file.txt  # Add ACL for a user
setfacl -x u:user file.txt      # Remove ACL for a user
```

- Useful for **shared directories** with multiple users needing different access levels.